

Selbststudium – Bicep einfach ausprobieren

In dieser Übung lernst du Bicep praktisch kennen.

Am Ende kannst du:

- eine einfache Bicep-Datei erstellen
- den Aufbau eines Bicep-Templates erkennen
- eine Azure-Ressource deklarativ beschreiben
- Fehler mit Unterstützung von Visual Studio Code erkennen

Zeitbedarf: ca. 45 Minuten

1. Visual Studio Code starten

Öffne **Visual Studio Code**.

Falls VS Code noch nicht installiert ist, installiere es zuerst.

Für diese Übung benötigst du **kein WSL und kein Ubuntu**.

2. Bicep-Erweiterung installieren

Öffne links in Visual Studio Code:

Extensions

oder drücke:

Ctrl + Shift + X

Suche nach:

Bicep

Installiere die Erweiterung:

Bicep – Microsoft

Nach der Installation kann Visual Studio Code Bicep-Dateien erkennen und unterstützt dich beim Schreiben des Codes.

3. Arbeitsordner erstellen

Erstelle auf deinem Computer einen neuen Ordner:

bicep-demo

Öffne diesen Ordner in Visual Studio Code:

File → Open Folder → bicep-demo

4. Bicep-Datei erstellen

Erstelle eine neue Datei mit dem Namen:

main.bicep

Wichtig ist die Dateiendung:

.bicep

5. Erste Ressource definieren

Füge folgenden Code in main.bicep ein:

```
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {  
  name: 'hfinfdemo12345'  
  location: 'switzerlandnorth'  
  
  sku: {  
    name: 'Standard_LRS'  
  }  
  
  kind: 'StorageV2'  
}
```

Speichere die Datei mit:

Ctrl + S

6. Code untersuchen

Versuche herauszufinden, was die einzelnen Angaben bedeuten.

resource storage

Wir definieren eine Ressource und geben ihr innerhalb des Templates den Namen storage.

Microsoft.Storage/storageAccounts

Es soll ein Azure Storage Account erstellt werden.

name

name: 'hfinfdemo12345'

Das ist der Name des Storage Accounts.

location

location: 'switzerlandnorth'

Die Ressource soll in der Azure-Region **Switzerland North** betrieben werden.

sku

```
sku: {  
  name: 'Standard_LRS'  
}
```

Hier wird unter anderem die Redundanz des Speichers festgelegt.

kind

kind: 'StorageV2'

Wir verwenden einen Storage Account vom Typ **StorageV2**.

7. Kleine Änderung durchführen

Ändere die Region:

location: 'westeuropa'

Beobachte Visual Studio Code.

Ändere sie danach wieder auf:

location: 'switzerlandnorth'

Damit siehst du einen wichtigen Vorteil von Infrastructure as Code:

Die gewünschte Infrastruktur wird durch Code beschrieben und kann einfach verändert werden.

8. Absichtlich einen Fehler erzeugen

Entferne beispielsweise das letzte } aus dem Template.

Visual Studio Code sollte nun einen Fehler anzeigen.

Setze die Klammer anschließend wieder ein.

Damit siehst du, dass dich die Entwicklungsumgebung bereits beim Erstellen der Infrastruktur unterstützt.

9. Manuell oder mit Bicep?

Stell dir vor, du müsstest denselben Storage Account manuell erstellen:

Azure Portal öffnen

→ Storage Account suchen

→ Create auswählen

→ Resource Group wählen

→ Name eingeben

→ Region auswählen

→ Redundanz konfigurieren

→ Einstellungen kontrollieren

→ Ressource erstellen

Mit Bicep steht die gewünschte Konfiguration dagegen direkt im Code.

```
resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
```

```
  name: 'hfinfdemo12345'
```

```
  location: 'switzerlandnorth'
```

```
  sku: {
```

```
    name: 'Standard_LRS'
```

```
  }
```

```
  kind: 'StorageV2'
```

```
}
```

10. Abschluss

Beantworte für dich folgende drei Fragen:

1. **Was wird mit resource beschrieben?**
2. **Warum ist Bicep deklarativ und nicht imperativ?**
3. **Welchen Vorteil hat eine Bicep-Datei gegenüber der manuellen Konfiguration im Azure Portal?**